

Deep Learning Practical 9

Neural networks as images, video and sound

We can think of images, audio and video as functions that map coordinates to values, such as pixel positions to colours or times to amplitudes. A continuous neural network can learn this mapping as an *implicit neural representation*, allowing the same fitted model to be sampled at different spatial resolutions, audio sampling rates or video frame rates.

1 Representing images

We will first fit the crow photograph by giving a network each pixel's position, (x, y) , and asking it to predict the corresponding red, green and blue values.

The code below downloads `multimodal.npz` from the research-data repository if it is not already in your working folder. The prepared arrays let us begin fitting without preprocessing. Run the blocks in order in a notebook, using NumPy, PyTorch, Matplotlib and IPython, with `imageio-ffmpeg` for video export.

```
import numpy as np
import torch
from torch import nn
import matplotlib.pyplot as plt
from pathlib import Path
from urllib.request import urlretrieve

device = "cuda" if torch.cuda.is_available() else "cpu"
url = "https://raw.githubusercontent.com/cwkw/research-data/main/datasets/multimodal.npz"
if not Path("multimodal.npz").exists():
    urlretrieve(url, "multimodal.npz")
data = np.load("multimodal.npz", allow_pickle=False)
image = data["crow"]
plt.imshow(image)
plt.axis("off")
plt.show()
```

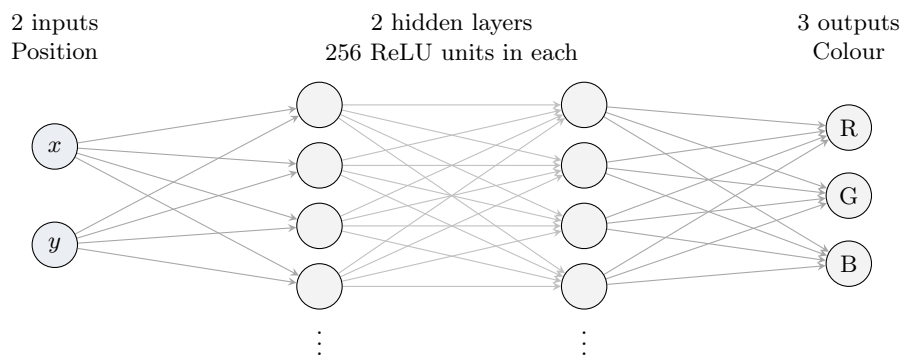


Figure 1: Two input neurons receive x and y , and three output neurons predict RGB. Only a few neurons are drawn in each hidden layer; the code uses 256 in each of two layers.

We can build this network from familiar linear layers and ReLU activations, leaving the output layer linear because we are predicting colour values rather than class probabilities.

```
def mlp(inputs, outputs):
    return nn.Sequential(
        nn.Linear(inputs, 256), nn.ReLU(),
        nn.Linear(256, 256), nn.ReLU(),
        nn.Linear(256, outputs))

torch.manual_seed(3801)
image_model = mlp(2, 3).to(device)
```

Choose pixels to learn from

When pixels are numbered row by row in an image of width W , pixel number i lies in column $i \bmod W$ and row $\lfloor i/W \rfloor$. Map each index to its pixel centre in $[-1, 1]$ using $2(j + \frac{1}{2})/N - 1$ for index j along an axis of length N .

```
def image_coordinates(index, shape):
    height, width = shape
    x = 2*((index % width).float()+0.5)/width-1
    y = 2*((index // width).float()+0.5)/height-1
    return torch.stack((x, y), dim=1)
```

At each update, select 4,096 pixels at random and minimise their mean squared colour error. To reuse the training loop for the other signals, the function below takes a model, an array of target values and a function that turns sample indices into coordinates. Integer RGB values are mapped from $[0, 255]$ to $[-1, 1]$ when a batch is selected.

```
def fit(model, array, coordinates, steps=2000):
    model.train()
    shape, channels = array.shape[:-1], array.shape[-1]
    values = torch.as_tensor(array.reshape(-1, channels), device=device)
    generator = torch.Generator(device=device).manual_seed(3802)
    optimiser = torch.optim.Adam(model.parameters(), lr=0.001)
    for step in range(steps):
        index = torch.randint(len(values), (4096,), device=device,
                               generator=generator)
        target = values[index].float()
        if values.dtype == torch.uint8:
            target = target/127.5-1
        prediction = model(coordinates(index, shape))
        loss = (prediction-target).square().mean()
        optimiser.zero_grad()
        loss.backward()
        optimiser.step()
        if (step+1) % 250 == 0:
            print(step+1, loss.item())
    return model.eval()
```

```
image_model = fit(image_model, image, image_coordinates)
```

Exercise

1. For one batch, what are the shapes of the network's input and output tensors? Why does the network receive two values per pixel rather than three?
2. Find the coordinates of the first and last pixels in the 1024×1024 crow image. Why must an index select both the position and its corresponding colour?
3. Why might the printed loss rise between two updates, even while the fit improves overall?

2 Reconstruct the crow image, then the parrot

To see the fitted image, query the network at every pixel centre and arrange its RGB outputs back into an image. We render in batches to keep the temporary tensors small and clip predictions to $[-1, 1]$ before displaying them, then map this range to $[0, 1]$ for Matplotlib.

```
@torch.no_grad()
def render(model, coordinates, shape, start=0, count=None):
    count = int(np.prod(shape)) if count is None else count
    output = []
    for first in range(0, count, 32768):
        last = min(first+32768, count)
        index = torch.arange(start+first, start+last, device=device)
        output.append(model(coordinates(index, shape)).clamp(-1, 1).cpu().numpy())
    return np.concatenate(output)
```

The same fitted network can also be sampled on a different grid without changing its weights. Render the crow at 32×32 and 512×512 , displaying both at the same size with nearest-neighbour interpolation so that the coarse grid's pixels remain visible:

```

fig, axes = plt.subplots(1, 3, figsize=(10, 4))
axes[0].imshow(image)
axes[0].set_title("Original")
for ax, size in zip(axes[1:], [32, 512]):
    rgb = render(image_model, image_coordinates, (size, size)).reshape(size, size, 3)
    ax.imshow((rgb+1)/2, interpolation="nearest")
    ax.set_title(f"Sampled at {size} x {size}")
for ax in axes:
    ax.axis("off")
plt.tight_layout()
plt.show()

```



Figure 2: The same crow model sampled at 32×32 and 512×512 , beside the original photograph. The smaller reconstruction is enlarged with nearest-neighbour interpolation to reveal its pixels, while the finer grid draws the learned function more smoothly without restoring missing feather detail.

To fit the parrot, change the target to its $(1356, 2040, 3)$ array and create a model with fresh weights. The coordinate function uses the new height and width, while the network keeps its two inputs and three outputs:

```

image = data["parrot"]
torch.manual_seed(3801)
image_model = fit(mlp(2, 3).to(device), image, image_coordinates)
reconstruction = render(image_model, image_coordinates, image.shape[:-1])
plt.imshow((reconstruction.reshape(image.shape)+1)/2)
plt.axis("off")
plt.show()

```

Exercise

1. Compare the outlines, colours and fine detail in both reconstructions. What has the network learned well, and what is missing?
2. Calculate the number of learned parameters in the two width-256 hidden layers and RGB output. Does it change when sampling the crow at a different resolution, or when fitting the larger parrot image?
3. Try a different hidden width or train for longer. Recreate the model and optimiser for each experiment, and compare the same parts of the image.

3 Represent videos by adding time

Because a video is a sequence of images, predicting the colour of a pixel now requires its position and its time: (x, y, t) . Adding this third input allows us to keep the same hidden layers and RGB output.

The cat array has shape $(272, 512, 512, 3)$, with axes for frame, row, column and channel, giving 10.88 seconds at 25 frames per second. Flat indices run through one frame before continuing into the next, so dividing by HW recovers the frame number. As with the image, the training loop samples across the complete array.

```

video = data["cat"]
fps = float(data["cat_fps"][0]/data["cat_fps"][1])

def video_coordinates(index, shape):
    frames, height, width = shape
    x = 2*((index % width).float()+0.5)/width-1
    y = 2*((index // width % height).float()+0.5)/height-1
    t = 2*((index // (height*width)).float()+0.5)/frames-1
    return torch.stack((x, y, t), dim=1)

torch.manual_seed(3801)
video_model = fit(mlp(3, 3).to(device), video, video_coordinates)

```

To see how well the network has learned the changing scene, render a few frames using their offsets within the full video grid:

```

frames, height, width, channels = video.shape
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
for column, k in enumerate([0, 90, 181, 271]):
    rgb = render(video_model, video_coordinates, video.shape[:-1],
                 start=k*height*width, count=height*width)
    axes[0, column].imshow(video[k])
    axes[1, column].imshow((rgb.reshape(height, width, 3)+1)/2)
    axes[0, column].set_title(f"{k/fps:.2f} s")
for row in axes:
    for ax in row:
        ax.set_xticks([])
        ax.set_yticks([])
axes[0, 0].set_ylabel("Original")
axes[1, 0].set_ylabel("ReLU")
plt.show()

```

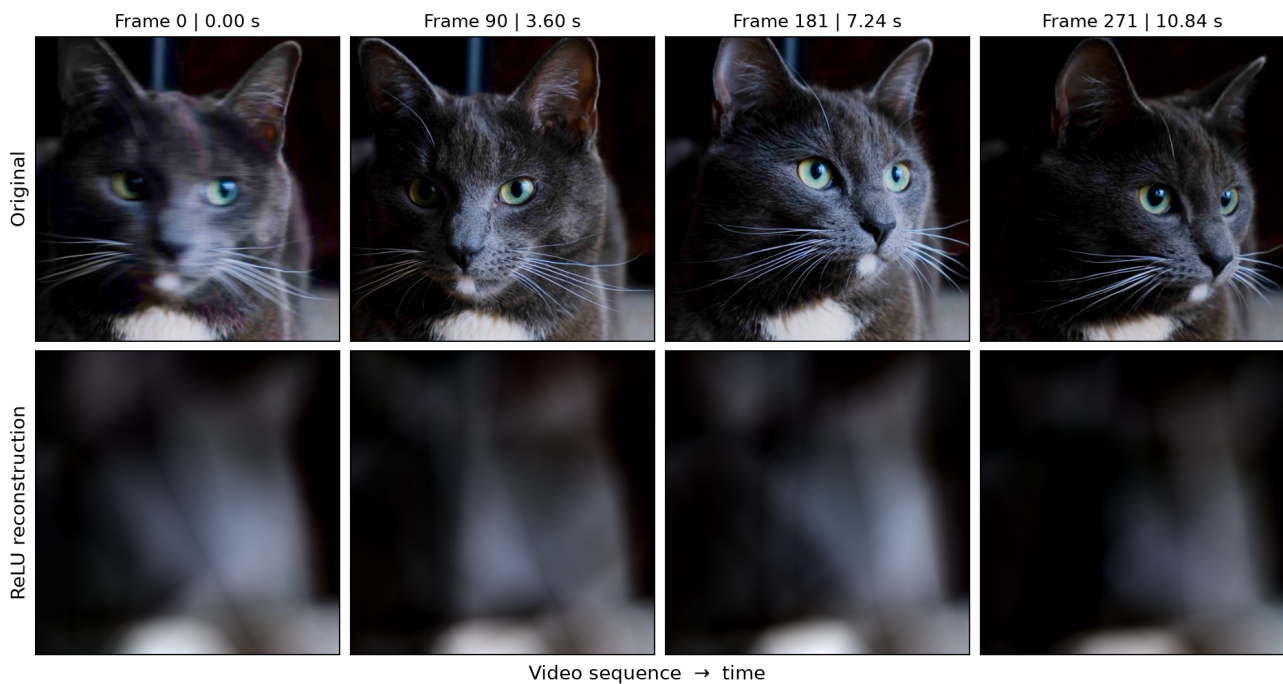


Figure 3: Four frames in time order from left to right, with the originals above their ReLU reconstructions.

To watch the reconstruction, render all frames into a compressed preview at the original frame rate. The H.264 settings below keep the upload small while preserving its dimensions and duration:

```

import imageio_ffmpeg
from IPython.display import Video, display

writer = imageio_ffmpeg.write_frames("cat-preview.mp4", (width, height), fps=fps,
    codec="libx264", pix_fmt_out="yuv420p", quality=None, macro_block_size=1,
    output_params=["-crf", "18", "-maxrate", "2500k", "-bufsize", "5000k",
        "-movflags", "+faststart"])
writer.send(None)
for k in range(frames):
    rgb = render(video_model, video_coordinates, video.shape[:-1],
        start=k*height*width, count=height*width)
    frame = np rint((rgb.reshape(height, width, 3)+1)*127.5).astype(np.uint8)
    writer.send(frame)
writer.close()
display(Video("cat-preview.mp4", embed=True))

```

Exercise

1. What would happen if every frame received the same time coordinate? Consider a pixel whose colour changes as the cat moves.
2. How many coordinates are there in the video, and why do we sample them in batches? How does its parameter count compare with the image model?
3. Watch the preview. Does it preserve motion as well as the broad appearance of the cat? What would happen to playback speed if you saved the same frames at 50 fps?

4 Representing sound

In Practical 7, we represented audio as a spectrogram and reconstructed a waveform afterwards. Here the network maps time directly to waveform amplitude, so we can sample its continuous output at any chosen audio rate. For this exercise, we fit and evaluate at the recording's original 44,100 Hz.

The Bach array holds (308207, 1) float32 samples already in $[-1, 1]$, so the training loop uses their amplitudes directly. Listen to the recording before defining its time coordinates:

```

from IPython.display import Audio

waveform = data["bach"]
rate = int(data["bach_sample_rate"])
display(Audio(waveform[:, 0], rate=rate, normalize=False))

def audio_coordinates(index, shape):
    return (2*(index.float()+0.5)/shape[0]-1).unsqueeze(1)

```

With time alone as input, the plain ReLU network learns a waveform close to silence because fitting rapid oscillations is much harder than fitting broad colour regions. We can help it by using a sine activation in the first hidden layer, followed by ReLU in the second. The input remains one time coordinate, and the output remains one amplitude.

In the model below, the first layer's weights determine how rapidly its sine units oscillate. At initialisation, we multiply successive rows of that weight matrix by factors ranging from 1 to 10,000, spaced evenly on a logarithmic scale, so that the units begin with both slow and fast variations. All weights are then learned as usual. Writing the layers and activations explicitly makes it easier to experiment with this choice:

```

class Model(nn.Module):
    def __init__(self, inputs=2, outputs=3, width=256):
        super().__init__()
        self.fc1 = nn.Linear(inputs, width)
        self.fc2 = nn.Linear(width, width)
        self.fc3 = nn.Linear(width, outputs)
        with torch.no_grad():
            self.fc1.weight *= torch.logspace(0, 4, width)[: , None]

    def forward(self, x):
        x = torch.sin(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

```

```

torch.manual_seed(3801)
audio_model = fit(Model(1, 1).to(device), waveform, audio_coordinates)
sound = render(audio_model, audio_coordinates, waveform.shape[:-1])[:, 0]
display(Audio(sound, rate=rate, normalize=False))

```

Plot the complete recording and reconstruction against time in seconds, keeping the same amplitude limits so that their changing amplitudes can be compared directly. Listening to both versions also reveals the distortion that remains despite the closer waveform fit.

```

time = np.arange(len(waveform))/rate
start, end = round(2*rate), round(2.05*rate)
fig, axes = plt.subplots(2, 1, figsize=(10, 4), sharex=True, sharey=True)
for ax, values, title in zip(axes, [waveform[:, 0], sound], ["Original", "Sine-ReLU reconstruction"]):
    ax.plot(time, values, linewidth=0.4)
    ax.set(title=title, ylabel="Amplitude", ylim=(-1, 1))
    inset = ax.inset_axes([.73, .57, .24, .37])
    inset.plot(time[start:end], values[start:end], linewidth=.65)
    inset.set(xlim=(2, 2.05), ylim=(-.65, .65), xticks=[2, 2.05], yticks=[])
    inset.set_title("50 ms detail", fontsize=9)
    inset.tick_params(labelsize=8)
axes[1].set_xlabel("Time (seconds)")
plt.tight_layout()
plt.show()

```

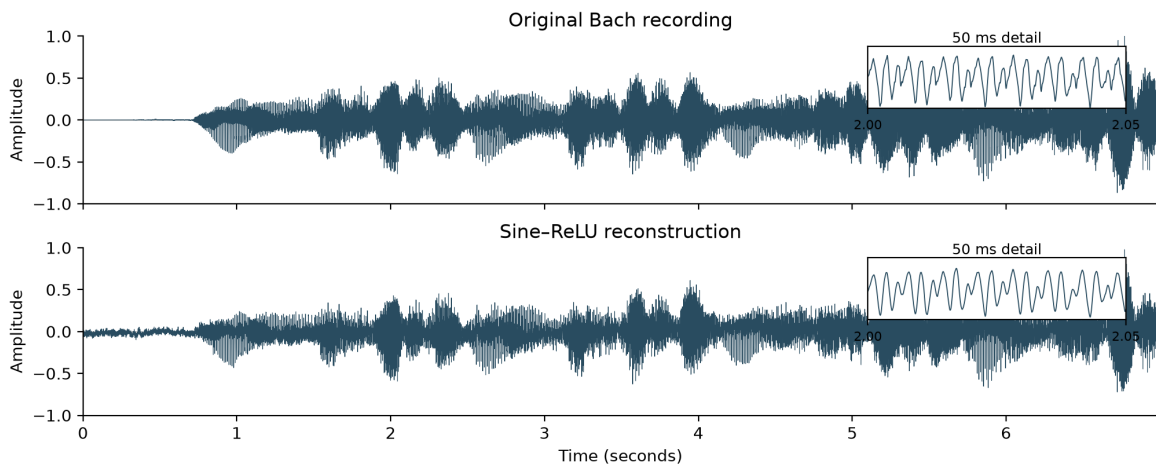


Figure 4: The original Bach waveform and sine-ReLU reconstruction on matching time and amplitude scales. Both insets enlarge 2.00–2.05 seconds with the same amplitude limits, so individual oscillations can be compared.

Exercise

1. Calculate the recording's duration. What would change in the network for stereo audio?
2. Compare this fit with `mlp(1, 1)` trained on `audio_coordinates`. Why does a sine first layer help, and why initialise its units at different frequency scales?
3. Why is it useful to keep the same vertical scale and to set `normalize=False` when listening to these two versions?

5 Improve your representation

The same model fits images with `Model(2, 3)`, video with `Model(3, 3)` and mono audio with `Model(1, 1)`, because only the coordinate and output dimensions change. Grid size determines where we sample the function. Fit parrot again and compare its detail with your earlier reconstruction:

```

torch.manual_seed(3801)
image_model = fit(Model(2, 3).to(device), image, image_coordinates)
reconstruction = render(image_model, image_coordinates, image.shape[:-1])
plt.imshow((reconstruction.reshape(image.shape)+1)/2)
plt.axis("off")
plt.show()

```

The submission program uses this model for all four signals, fitting fresh weights each time. Its loading, training, scoring and export code is contained in one Python file.

For a numerical comparison, compute mean squared error over every coordinate. With targets and clipped predictions in $[-1, 1]$, use $\text{PSNR} = 10 \log_{10}(4/\text{MSE})$. This gives a higher PSNR for a smaller reconstruction error, as you can check for the parrot reconstruction:

```
target = image.astype(np.float32)/127.5-1
mse = np.mean((reconstruction.reshape(image.shape)-target)**2)
print("Parrot PSNR:", 10*np.log10(4/max(mse, 1e-12)), "dB")
```

The challenge allows **1 MiB (1,048,576 bytes) of saved model state per signal**. The submission program counts all weights, biases, stored features and file headers, then reloads the saved values for evaluation on the original grids. For video, it computes one MSE over all frames and channels before converting it to PSNR, using predictions before rounding or video compression. The compressed MP4 is a preview and does not determine the score.

With seed 3801, both fits use 2,000 updates per signal, batches of 4,096 and learning rate 0.001, giving the full-grid scores below. Aim to improve on the sine-ReLU submission baseline while staying within the storage allowance.

Signal	Plain ReLU (dB)	Sine-ReLU (dB)
Parrot	19.02	25.81
Crow	19.45	27.70
Bach	22.18	31.94
Cat	19.46	21.50

Exercise

1. Compare one change to width, depth, learning rate or training time. Keep the other settings fixed, and explain the change in both the reconstruction and its score.
2. How many float32 values fit in 1 MiB before file headers are included? Why does a larger image not necessarily require a larger network?
3. A silent Bach reconstruction has a PSNR of about 22.18 dB. Why can this be a poor description of audible quality? Is this task measuring reconstruction or prediction on unseen data?

6 What is possible?

The parrot reconstructions below show how different representations retain finer detail within the same storage allowance, particularly around the eye. SIREN uses sine hidden activations with a matching initialisation, while multiresolution hash grids learn features at several scales.



Figure 5: Original and fitted images above, with the marked eye region enlarged below. All models were trained on the full image, and each fitted representation occupies less than 1 MiB.

For a conventional SIREN, use raw coordinates and the paper’s weight initialisation, since replacing the remaining ReLU with sine alone may give disappointing results. The references below provide a starting point.

Further reading. Max Slater, Exploring Neural Graphics Primitives, an illustrated introduction to coordinate networks, activations and input encodings; Sitzmann et al., Implicit Neural Representations with Periodic Activation Functions.

7 Submission

Download `d1-practical9.py` [↗](#). It downloads `multimodal.npz` [↗](#) from the research-data repository on first use. Set `STUDENT_ID` in `d1-practical9.py` [↗](#) and edit its model and training loop. Run the complete file, or paste it into one notebook cell. It loads the data, scores and exports the reconstructions, and creates the ZIP below. Upload this ZIP to the corresponding Ultra item.

```
lab-09-studentID.zip
lab-09-code.py
lab-09-results.json
samples/
  parrot.png
  crow.png
  bach.wav
  cat.mp4
models/
  parrot.npz
  crow.npz
  bach.npz
  cat.npz
```

Details and constraints

- Save `parrot.png` and `crow.png` at their original dimensions, `bach.wav` at 44,100 Hz with all 308,207 samples, and `cat.mp4` with all 272 colour frames at 512 by 512 and 25 frames per second. Use a compressed H.264 preview below 5 MB (5,000,000 bytes), as produced by the program.
- Each saved model NPZ must be at most 1,048,576 bytes (1 MiB), including all parameters, feature tables, buffers and archive headers. Use the same model design and training settings for all four fits, with fresh weights each time. Input and output dimensions follow the signal shape. Record the batch size and number of updates; these have no fixed cap.
- Use the full original grids in `multimodal.npz`. Draw training coordinates uniformly across each complete signal, and reconstruct in batches without cropping, resizing, dropping frames or resampling the waveform.
- At reconstruction time, use only coordinates, public shape/rate metadata and the counted model state. Do not read the targets in the model, hide signal values in source code, use pretrained weights, or use different model settings for named files. Learned feature tables are permitted.
- Score every original coordinate after reloading the saved model. Targets and clipped predictions lie in $[-1, 1]$, so PSNR is $10 \log_{10}(4/\text{MSE})$. Report all four PSNRs, their equally weighted mean, and waveform SNR. Scores use network predictions before rounding or video compression, rather than pixels decoded from the MP4 preview.
- Put a brief abstract at the top of the Python file, with references and AI acknowledgements in comments. The program writes the JSON and sample files; submit these unchanged.

Marking

- Marks reflect the quality and simplicity of your solution and its measured performance within the constraints. Aim for higher PSNR than the sine-ReLU baseline on each of `parrot`, `crow`, `Bach` and `cat`.
- A pass requires a small improvement over the baseline. A 2:1 requires significant improvements using established techniques discussed in the course. A high first requires substantial improvements through well-implemented state-of-the-art methods, for example by integrating methods from recent papers, producing high-quality samples and scores within the budget.
- The exact marking function is withheld to avoid gamification.

Solutions for this assignment cannot later be publicised without request. Requests will be considered on a case-by-case basis after marking.